

Project Title

JavaScript Logic & State Management To-Do List Application

Objective

The objective of this project is to develop an interactive and responsive client-side To-Do List application using JavaScript. The project focuses on mastering DOM manipulation, event handling, CRUD operations, filtering functionality, and browser local storage for persistent data management.

Tools Used

- HTML5
- CSS3
- JavaScript
- Visual Studio Code / Notepad
- Google Chrome Browser
- Local Storage API

Dataset Description

No external dataset is used in this project. User-generated task data such as task names and completion status are stored dynamically using `window.localStorage`.

Machine Learning Algorithms Used

No machine learning algorithms are used in this project because it is based on frontend JavaScript logic and state management.

Key Features

- Full CRUD Operations
- Add New Tasks

- Edit Existing Tasks
- Delete Tasks
- Mark Tasks as Completed
- Local Storage Data Persistence
- Advanced Filtering (All, Active, Completed)
- Dynamic DOM Manipulation
- Event Listener Handling
- Professional Responsive UI Design

Steps Performed

1. Created the basic HTML structure for the To-Do application.
2. Designed a professional responsive UI using CSS3.
3. Implemented JavaScript logic for task creation and deletion.
4. Added update/edit functionality for existing tasks.
5. Implemented task completion toggling.
6. Stored task data using `window.localStorage`.
7. Developed filter buttons for All, Active, and Completed tasks.
8. Used dynamic DOM manipulation and event listeners.
9. Tested application functionality and responsiveness.

Output

- Successfully developed a fully functional To-Do List application.
- Implemented CRUD operations with dynamic task management.
- Added persistent local storage functionality.
- Created responsive and professional task management UI.
- Enabled task filtering and completion tracking features.

Sample HTML, CSS & JavaScript Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
```

```
<title>Professional To-Do App</title>
```

```
<style>
```

```
*{
```

```
  margin:0;
```

```
  padding:0;
```

```
  box-sizing:border-box;
```

```
  font-family:Arial,sans-serif;
```

```
}
```

```
body{
```

```
  background:linear-gradient(to right,#141e30,#243b55);
```

```
  min-height:100vh;
```

```
  display:flex;
```

```
  justify-content:center;
```

```
  align-items:center;
```

```
  color:white;
```

```
}
```

```
.container{
```

```
  width:90%;
```

```
  max-width:500px;
```

```
  background:rgba(255,255,255,0.1);
```

```
  padding:30px;
```

```
  border-radius:20px;
```

```
  backdrop-filter:blur(10px);
```

```
  box-shadow:0 10px 25px rgba(0,0,0,0.3);
```

```
}
```

```
h1{
```

```
  text-align:center;
```

```
  margin-bottom:20px;
```

```
  color:#00d4ff;
```

```
}
```

```
.input-section{
```

```
  display:flex;
```

```
  gap:10px;
```

```
}
```

```
input{
```

```
    flex:1;
    padding:12px;
    border:none;
    border-radius:8px;
    outline:none;
}

button{
    padding:12px 18px;
    border:none;
    border-radius:8px;
    cursor:pointer;
    background:#00d4ff;
    color:black;
    font-weight:bold;
}

.filters{
    display:flex;
    justify-content:center;
    gap:10px;
    margin-top:20px;
}

.filters button{
    background:#1e90ff;
    color:white;
}

ul{
    list-style:none;
    margin-top:20px;
}

li{
    background:rgba(255,255,255,0.1);
    padding:15px;
    border-radius:10px;
    margin-top:10px;
    display:flex;
    justify-content:space-between;
    align-items:center;
```

```
}

.completed{
  text-decoration:line-through;
  opacity:0.6;
}

.task-buttons button{
  margin-left:5px;
  padding:8px 10px;
}

.edit-btn{
  background:orange;
}

.delete-btn{
  background:red;
  color:white;
}

.complete-btn{
  background:green;
  color:white;
}

@media(max-width:600px){

  .input-section{
    flex-direction:column;
  }

  button{
    width:100%;
  }
}

</style>
</head>

<body>
```

```
<div class="container">

  <h1>To-Do List App</h1>

  <div class="input-section">

    <input type="text" id="taskInput" placeholder="Enter a
task">

    <button onclick="addTask()">Add</button>

  </div>

  <div class="filters">

    <button onclick="filterTasks('all')">All</button>

    <button onclick="filterTasks('active')">Active</button>

    <button
onclick="filterTasks('completed')">Completed</button>

  </div>

  <ul id="taskList"></ul>

</div>

<script>

let tasks = JSON.parse(localStorage.getItem("tasks")) || [];

function saveTasks(){
  localStorage.setItem("tasks", JSON.stringify(tasks));
}

function renderTasks(filter = "all"){

  const taskList = document.getElementById("taskList");

  taskList.innerHTML = "";
```

```

let filteredTasks = tasks.filter(task => {

    if(filter === "active") return !task.completed;

    if(filter === "completed") return task.completed;

    return true;
});

filteredTasks.forEach((task,index)=>{

    let li = document.createElement("li");

    li.className = task.completed ? "completed" : "";

    li.innerHTML = `

        <span>${task.text}</span>

        <div class="task-buttons">

            <button class="complete-btn"
            onclick="toggleComplete(${index})">

                ✓

            </button>

            <button class="edit-btn"
            onclick="editTask(${index})">

                Edit

            </button>

            <button class="delete-btn"
            onclick="deleteTask(${index})">

                Delete

            </button>

```

```

        </div>
    `;

    taskList.appendChild(li);

});

}

function addTask(){

    let input = document.getElementById("taskInput");

    if(input.value.trim() === "") return;

    tasks.push({
        text:input.value,
        completed:false
    });

    saveTasks();

    renderTasks();

    input.value = "";

}

function deleteTask(index){

    tasks.splice(index,1);

    saveTasks();

    renderTasks();

}

function toggleComplete(index){

    tasks[index].completed = !tasks[index].completed;

```



```
        saveTasks();

        renderTasks();
    }

    function editTask(index){

        let updatedTask = prompt("Edit Task", tasks[index].text);

        if(updatedTask !== null){

            tasks[index].text = updatedTask;

            saveTasks();

            renderTasks();
        }
    }

    function filterTasks(filter){

        renderTasks(filter);

    }

    renderTasks();

</script>

</body>
</html>
```

Conclusion

This project successfully demonstrates JavaScript logic and state management concepts by implementing a fully interactive To-Do List application. The project effectively uses CRUD operations, local storage, DOM manipulation, and event handling to provide a dynamic and responsive user experience.

Expected Outcome

- Fully functional task management application
- Persistent task storage using localStorage
- Dynamic CRUD operations
- Responsive and professional UI design
- Advanced task filtering system
- Improved JavaScript DOM manipulation skills

OUTPUT

